Functional dependencies

`X → Y` means that X uniquely determines Y.

Table	Main functional dependencies	Candidate key(s)
`circles`	`circle_id → name, contribution_amount, contact_email`; `name → circle_id`	`circle_id`, `name`
`members`	`member_id → full_name, phone, email, account_status`	`member_id`
`memberships`	`membership_id → circle_id, member_id, rotation_position, status`; `(circle_id, rotation_position) → membership_id, member_id, status`	`membership_id`, `(circle_id, rotation_position)`
`cycles`	`cycle_id → circle_id, cycle_number`; `(circle_id, cycle_number) → cycle_id`	`cycle_id`, `(circle_id, cycle_number)`
`contributions` (v1)	`contribution_id → membership_id, cycle_id, amount, contribution_date, payment_status`; `(membership_id, cycle_id) → contribution_id, amount, contribution_date, payment_status`	`contribution_id`, `(membership_id, cycle_id)`
`payouts`	`payout_id → cycle_id, membership_id, amount, payout_date, payout_status`; `cycle_id → payout_id, membership_id, amount, payout_date, payout_status`	`payout_id`, `cycle_id`

Normal-form audit

- 1NF: All columns contain single, atomic values.
- 2NF: Non-key attributes depend on whole candidate keys; no partial dependencies were identified.
- 3NF: Under the fixed-contribution rule, `cycle_id → amount` in `contributions`, because each cycle belongs to a circle and each circle has one fixed contribution amount. `cycle_id` is not a candidate key of contributions, so this creates a non-key dependency and duplicates data.
- BCNF: The same dependency violates BCNF because `cycle_id` is not a superkey of `contributions`. The other listed dependencies have candidate keys as determinants.

Revised design

Remove `contributions.amount` and keep the authoritative fixed amount in `circles.contribution_amount`. The `contribution_details` view joins the tables to expose the amount for reports and existing read queries.

The migration checks that existing contribution amounts match the circle's fixed amount before removing the redundant column. It preserves contribution and payout history. `payouts.amount` remains because it records the actual disbursement.

Constraints and performance

Primary/foreign keys and UNIQUE constraints enforce entity identity, cycle numbers, rotation positions, one contribution per membership/cycle, and one payout per cycle. Triggers enforce cross-table business rules, including valid circle membership and payout rotation.

Indexes support membership status lookups, cycle contribution summaries, contribution history, and payout history. Partitioning is not justified by the small supplied dataset; further indexes should be added only after reviewing production query plans.